

# ArtConnect — Livrable Étape 4

Intégration de la base dans l'application Java

Bases de données 2 — TI603 — INGE1 — EFREI Paris 2025/2026

## 1. Architecture de l'application

L'application ArtConnectPro suit une architecture en 4 couches strictement séparées. Chaque couche ne communique qu'avec la couche immédiatement inférieure, ce qui garantit l'indépendance des composants et la testabilité du code.

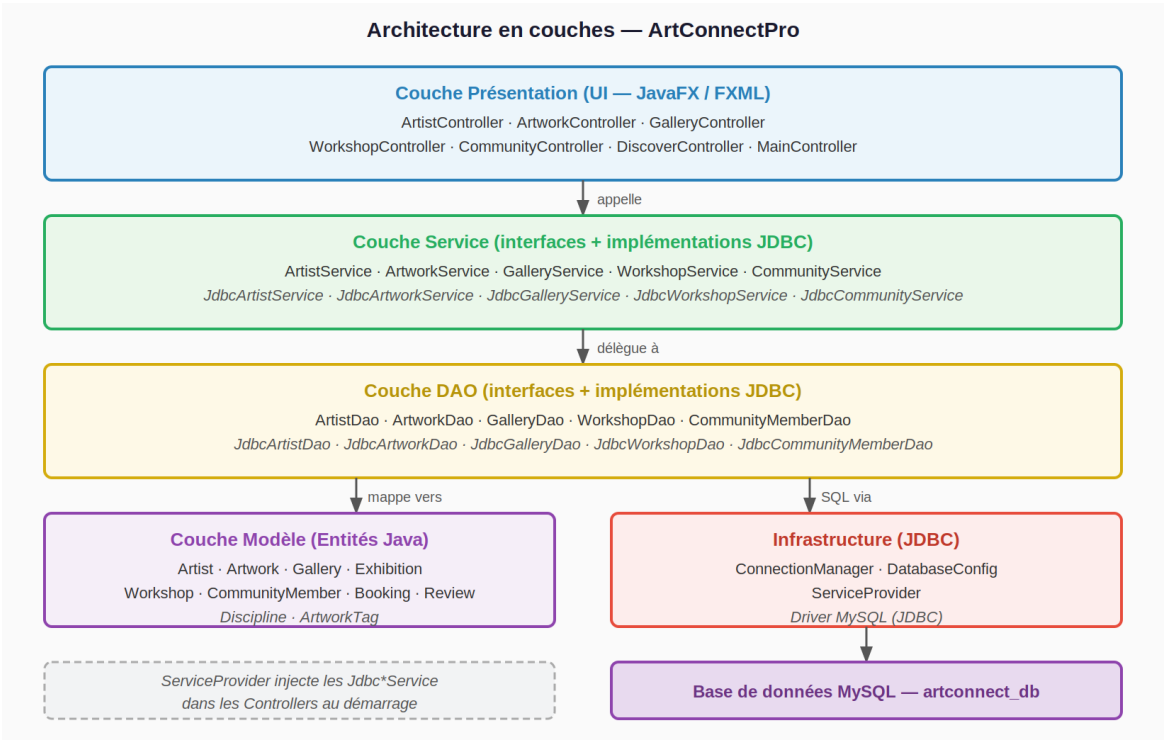


Figure 1 — Schéma en couches — ArtConnectPro

### 1.1 Description des couches

| Couche            | Classes principales                                                                                                 | Rôle                                                                  |
|-------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Présentation (UI) | ArtistController, ArtworkController, GalleryController, WorkshopController, CommunityController, DiscoverController | Affichage JavaFX / FXML. Appelle les services. Aucune logique métier. |

| Couche         | Classes principales                                                                                               | Rôle                                                                             |
|----------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Service        | JdbcArtistService,<br>JdbcArtworkService,<br>JdbcGalleryService,<br>JdbcWorkshopService,<br>JdbcCommunityService  | Logique métier et orchestration. Implémentent les interfaces ArtistService, etc. |
| DAO            | JdbcArtistDao, JdbcArtworkDao,<br>JdbcGalleryDao,<br>JdbcWorkshopDao,<br>JdbcCommunityMemberDao                   | Accès à la base via JDBC. PreparedStatement, gestion des ressources.             |
| Modèle         | Artist, Artwork, Gallery,<br>Exhibition, Workshop,<br>CommunityMember, Booking,<br>Review, Discipline, ArtworkTag | Entités Java simples (POJO) avec id Long et getters/setters.                     |
| Infrastructure | ConnectionManager,<br>DatabaseConfig,<br>ServiceProvider                                                          | Connexion JDBC centralisée. Injection des services dans les controllers.         |

## 1.2 Flux d'une requête type

Exemple : affichage de la liste des artistes dans l'UI.

- 1. ArtistController.initialize() appelle ServiceProvider.getArtistService()
- 2. JdbcArtistService.getAllArtists() délègue à JdbcArtistDao.findAll()
- 3. JdbcArtistDao ouvre une connexion via ConnectionManager.getConnection()
- 4. Exécution du SELECT sur la table artist, mappage des ResultSet vers des objets Artist
- 5. La liste remonte jusqu'au Controller qui peuple le TableView JavaFX

## 2. Code des entités (Modèle)

Les entités sont des classes Java simples. Un champ id (Long, AUTO\_INCREMENT côté MySQL) a été ajouté à chaque entité par rapport au squelette original.

### 2.1 Artist.java

```
package com.project.artconnect.model;

import java.util.ArrayList;
import java.util.List;

public class Artist {
    private Long id;
    private String name;
    private String bio;
    private Integer birthYear;
    private String contactEmail;
    private String phone;
    private String city;
    private String website;
    private String socialMedia;
    private boolean isActive;
    private List<Discipline> disciplines = new ArrayList<>();
    private List<Artwork> artworks = new ArrayList<>();

    // Getters / Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getBio() { return bio; }
    public void setBio(String bio) { this.bio = bio; }
    public Integer getBirthYear() { return birthYear; }
    public void setBirthYear(Integer y) { this.birthYear = y; }
    public String getContactEmail() { return contactEmail; }
    public void setContactEmail(String e) { this.contactEmail = e; }
    public String getPhone() { return phone; }
    public void setPhone(String phone) { this.phone = phone; }
    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }
    public String getWebsite() { return website; }
    public void setWebsite(String w) { this.website = w; }
    public String getSocialMedia() { return socialMedia; }
    public void setSocialMedia(String s) { this.socialMedia = s; }
    public boolean isActive() { return isActive; }
    public void setActive(boolean a) { this.isActive = a; }
    public List<Discipline> getDisciplines() { return disciplines; }
    public void setDisciplines(List<Discipline> d) { this.disciplines = d; }
    public List<Artwork> getArtworks() { return artworks; }
    public void addArtwork(Artwork a) { this.artworks.add(a); }

    @Override
    public String toString() { return name; }
}
```

### 2.2 Artwork.java (extrait)

```
public class Artwork {
    private Long id;
    private String title;
```

```

private Integer creationYear;
private String  type;
private String  medium;
private String  dimensions;
private String  description;
private double  price;
private Status  status;
private Artist  artist;
private List<ArtworkTag> tags = new ArrayList<>();

public enum Status { FOR_SALE, SOLD, EXHIBITED }

// id, title, creationYear, type, medium, dimensions,
// description, price, status, artist - getters/setters omis pour brièveté
public Long  getId()      { return id; }
public void  setId(Long id) { this.id = id; }
public String getTitle()  { return title; }
public void  setTitle(String t) { this.title = t; }
public Artist getArtist() { return artist; }
public void  setArtist(Artist a) { this.artist = a; }
public Status getStatus() { return status; }
public void  setStatus(Status s) { this.status = s; }
// ...
}

```

Les autres entités (Gallery, Exhibition, Workshop, CommunityMember, Booking, Review) suivent le même patron : id Long + champs métier + getters/setters. Chaque entité embarquant ses associations (ex. Gallery contient List<Exhibition>).

### 3. Code des DAOs

Chaque DAO implémente l'interface correspondante. Les principes communs :  
PreparedStatement pour éviter les injections SQL, try-with-resources pour fermer automatiquement connexions et curseurs, batch insert pour les relations many-to-many.

#### 3.1 ConnectionManager.java

```
package com.project.artconnect.util;

import com.project.artconnect.config.DatabaseConfig;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConnectionManager {

    private ConnectionManager() {}

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(
            DatabaseConfig.URL,
            DatabaseConfig.USER,
            DatabaseConfig.PASSWORD
        );
    }
}
```

#### 3.2 DatabaseConfig.java

```
package com.project.artconnect.config;

public class DatabaseConfig {
    public static final String URL = "jdbc:mysql://localhost:3306/artconnect_db"
    + "?useSSL=false&serverTimezone=UTC";
    public static final String USER = "root";
    public static final String PASSWORD = "votre_mot_de_passe";
}
```

#### 3.3 JdbcArtistDao.java

```
package com.project.artconnect.persistence;

import com.project.artconnect.dao.ArtistDao;
import com.project.artconnect.model.Artist;
import com.project.artconnect.model.Discipline;
import com.project.artconnect.util.ConnectionManager;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class JdbcArtistDao implements ArtistDao {

    @Override
    public List<Artist> findAll() {
```

```

List<Artist> artists = new ArrayList<>();
String sql = "SELECT id, name, bio, birth_year, contact_email, phone, " +
    "city, website, social_media, is_active FROM artist";
try (Connection conn = ConnectionManager.getConnection();
    Statement st = conn.createStatement();
    ResultSet rs = st.executeQuery(sql)) {
    while (rs.next()) {
        Artist a = mapRow(rs);
        a.setDisciplines(findDisciplinesByArtistId(conn, a.getId()));
        artists.add(a);
    }
} catch (SQLException e) {
    throw new RuntimeException("Erreur findAll artist", e);
}
return artists;
}

@Override
public void save(Artist artist) {
    String sql = "INSERT INTO artist (name, bio, birth_year, contact_email, " +
        "phone, city, website, social_media, is_active) " +
        "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";
    try (Connection conn = ConnectionManager.getConnection();
        PreparedStatement ps = conn.prepareStatement(
            sql, Statement.RETURN_GENERATED_KEYS)) {
        setArtistParams(ps, artist);
        ps.executeUpdate();
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) artist.setId(keys.getLong(1));
        }
        saveDisciplines(conn, artist);
    } catch (SQLException e) {
        throw new RuntimeException("Erreur save artist : " + artist.getName(), e);
    }
}

@Override
public void update(Artist artist) {
    String sql = "UPDATE artist SET bio=?, birth_year=?, contact_email=?, " +
        "phone=?, city=?, website=?, social_media=?, is_active=? " +
        "WHERE name=?";
    try (Connection conn = ConnectionManager.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, artist.getBio());
        ps.setObject(2, artist.getBirthYear());
        ps.setString(3, artist.getContactEmail());
        ps.setString(4, artist.getPhone());
        ps.setString(5, artist.getCity());
        ps.setString(6, artist.getWebsite());
        ps.setString(7, artist.getSocialMedia());
        ps.setBoolean(8, artist.isActive());
        ps.setString(9, artist.getName());
        ps.executeUpdate();
        // resync disciplines
        try (PreparedStatement del = conn.prepareStatement(
            "DELETE FROM artist_discipline WHERE artist_id = ?")) {
            del.setLong(1, artist.getId());
            del.executeUpdate();
        }
        saveDisciplines(conn, artist);
    } catch (SQLException e) {
        throw new RuntimeException("Erreur update artist : " + artist.getName(), e);
    }
}

@Override
public void delete(String name) {
    try (Connection conn = ConnectionManager.getConnection();
        PreparedStatement ps = conn.prepareStatement(
            "DELETE FROM artist WHERE name = ?")) {

```

```

        ps.setString(1, name);
        ps.executeUpdate();
    } catch (SQLException e) {
        throw new RuntimeException("Erreur delete artist : " + name, e);
    }
}

@Override
public List<Artist> findByCity(String city) {
    List<Artist> artists = new ArrayList<>();
    String sql = "SELECT id, name, bio, birth_year, contact_email, phone, " +
        "city, website, social_media, is_active FROM artist WHERE city =
?";

    try (Connection conn = ConnectionManager.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, city);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                Artist a = mapRow(rs);
                a.setDisciplines(findDisciplinesByArtistId(conn, a.getId()));
                artists.add(a);
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException("Erreur findByCity : " + city, e);
    }
    return artists;
}

// — helpers privés —————
private Artist mapRow(ResultSet rs) throws SQLException {
    Artist a = new Artist();
    a.setId(rs.getLong("id"));
    a.setName(rs.getString("name"));
    a.setBio(rs.getString("bio"));
    a.setBirthYear(rs.getObject("birth_year", Integer.class));
    a.setContactEmail(rs.getString("contact_email"));
    a.setPhone(rs.getString("phone"));
    a.setCity(rs.getString("city"));
    a.setWebsite(rs.getString("website"));
    a.setSocialMedia(rs.getString("social_media"));
    a.setActive(rs.getBoolean("is_active"));
    return a;
}

private void setArtistParams(PreparedStatement ps, Artist a) throws SQLException {
    ps.setString(1, a.getName());
    ps.setString(2, a.getBio());
    ps.setObject(3, a.getBirthYear());
    ps.setString(4, a.getContactEmail());
    ps.setString(5, a.getPhone());
    ps.setString(6, a.getCity());
    ps.setString(7, a.getWebsite());
    ps.setString(8, a.getSocialMedia());
    ps.setBoolean(9, a.isActive());
}

private List<Discipline> findDisciplinesByArtistId(
    Connection conn, Long artistId) throws SQLException {
    List<Discipline> list = new ArrayList<>();
    String sql = "SELECT d.name FROM discipline d "
        + "JOIN artist_discipline ad ON d.id = ad.discipline_id "
        + "WHERE ad.artist_id = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, artistId);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(new Discipline(rs.getString("name")));
        }
    }
}

```

```
        return list;
    }

    private void saveDisciplines(Connection conn, Artist artist) throws SQLException {
        if (artist.getId() == null || artist.getDisciplines() == null) return;
        String sql = "INSERT IGNORE INTO artist_discipline (artist_id, discipline_id) "
            + "SELECT ?, id FROM discipline WHERE name = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            for (Discipline d : artist.getDisciplines()) {
                ps.setLong(1, artist.getId());
                ps.setString(2, d.getName());
                ps.addBatch();
            }
            ps.executeBatch();
        }
    }
}
```

### 3.4 JdbcArtworkDao.java

```
public class JdbcArtworkDao implements ArtworkDao {

    @Override
    public List<Artwork> findAll() {
        List<Artwork> list = new ArrayList<>();
        String sql = "SELECT aw.id, aw.title, aw.creation_year, aw.type, aw.medium, " +
            "aw.dimensions, aw.description, aw.price, aw.status, " +
            "ar.id AS artist_id, ar.name AS artist_name " +
            "FROM artwork aw JOIN artist ar ON aw.artist_id = ar.id";
        try (Connection conn = ConnectionManager.getConnection();
            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) list.add(mapRow(rs));
        } catch (SQLException e) {
            throw new RuntimeException("Erreur findAll artwork", e);
        }
        return list;
    }

    @Override
    public void save(Artwork artwork) {
        String sql = "INSERT INTO artwork (title, creation_year, type, medium,
dimensions, " +
            "description, price, status, artist_id) VALUES (?, ?, ?, ?, ?, ?, ?,
?, ?, ?)";
        try (Connection conn = ConnectionManager.getConnection();
            PreparedStatement ps = conn.prepareStatement(
                sql, Statement.RETURN_GENERATED_KEYS)) {
            setParams(ps, artwork);
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (keys.next()) artwork.setId(keys.getLong(1));
            }
        } catch (SQLException e) {
            throw new RuntimeException("Erreur save artwork : " + artwork.getTitle(),
e);
        }
    }

    @Override
    public void delete(String title) {
        try (Connection conn = ConnectionManager.getConnection();
            PreparedStatement ps = conn.prepareStatement(
                "DELETE FROM artwork WHERE title = ?")) {
            ps.setString(1, title);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new RuntimeException("Erreur delete artwork : " + title, e);
        }
    }

    private Artwork mapRow(ResultSet rs) throws SQLException {
        Artwork a = new Artwork();
        a.setId(rs.getLong("id"));
        a.setTitle(rs.getString("title"));
        a.setCreationYear(rs.getObject("creation_year", Integer.class));
        a.setType(rs.getString("type"));
        a.setMedium(rs.getString("medium"));
        a.setDimensions(rs.getString("dimensions"));
        a.setDescription(rs.getString("description"));
        a.setPrice(rs.getDouble("price"));
        String status = rs.getString("status");
        if (status != null) a.setStatus(Artwork.Status.valueOf(status));
        Artist artist = new Artist();
        artist.setId(rs.getLong("artist_id"));
        artist.setName(rs.getString("artist_name"));
        a.setArtist(artist);
    }
}
```

```
        return a;
    }
    // update, findByArtistName, setParams – même pattern que JdbcArtistDao
}
```

Les DAOs `JdbcGalleryDao`, `JdbcWorkshopDao` et `JdbcCommunityMemberDao` suivent le même patron. Les opérations de lecture chargent les entités associées (ex. Gallery charge ses Exhibition, qui chargent leurs Artwork) via des requêtes sur la même connexion ouverte.

## 4. Code des services

Les services JDBC implémentent les mêmes interfaces que les anciens `InMemory*Service`, garantissant ainsi que les contrôleurs JavaFX n'ont pas eu besoin d'être modifiés.

### 4.1 ServiceProvider.java

```
package com.project.artconnect.util;

import com.project.artconnect.service.*;
import com.project.artconnect.service.impl.*;

/**
 * Fournit des instances singleton des services.
 * Remplace les InMemory* par les implémentations JDBC.
 */
public class ServiceProvider {

    private static final ArtistService    artistService    = new JdbcArtistService();
    private static final ArtworkService   artworkService   = new JdbcArtworkService();
    private static final GalleryService   galleryService    = new JdbcGalleryService();
    private static final WorkshopService  workshopService  = new JdbcWorkshopService();
    private static final CommunityService communityService  = new JdbcCommunityService();

    public static ArtistService    getArtistService()    { return artistService; }
    public static ArtworkService   getArtworkService()   { return artworkService; }
    public static GalleryService   getGalleryService()   { return galleryService; }
    public static WorkshopService  getWorkshopService()  { return workshopService; }
    public static CommunityService getCommunityService() { return communityService; }
}
```

### 4.2 JdbcArtistService.java

```
package com.project.artconnect.service.impl;

import com.project.artconnect.model.Artist;
import com.project.artconnect.model.Discipline;
import com.project.artconnect.persistence.JdbcArtistDao;
import com.project.artconnect.service.ArtistService;
import com.project.artconnect.util.ConnectionManager;
import java.sql.*;
import java.util.*;
import java.util.stream.Collectors;

public class JdbcArtistService implements ArtistService {

    private final JdbcArtistDao artistDao = new JdbcArtistDao();

    @Override
    public List<Artist> getAllArtists() {
        return artistDao.findAll();
    }

    @Override
    public Optional<Artist> getArtistByName(String name) {
        return artistDao.findAll().stream()
            .filter(a -> a.getName().equals(name))
            .findFirst();
    }
}
```

```

@Override
public void createArtist(Artist artist) {
    artistDao.save(artist);
}

@Override
public void updateArtist(Artist artist) {
    artistDao.update(artist);
}

@Override
public void deleteArtist(String name) {
    artistDao.delete(name);
}

@Override
public List<Discipline> getAllDisciplines() {
    String sql = "SELECT name FROM discipline ORDER BY name";
    List<Discipline> list = new ArrayList<>();
    try (Connection conn = ConnectionManager.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) list.add(new Discipline(rs.getString("name")));
    } catch (SQLException e) {
        throw new RuntimeException("Erreur getAllDisciplines", e);
    }
    return list;
}

@Override
public List<Artist> searchArtists(String query, String discipline, String city) {
    return artistDao.findAll().stream()
        .filter(a -> query == null || query.isEmpty()
            || a.getName().toLowerCase().contains(query.toLowerCase()))
        .filter(a -> city == null || city.isEmpty()
            || a.getCity().equalsIgnoreCase(city))
        .filter(a -> discipline == null || discipline.isEmpty()
            || a.getDisciplines().stream()
                .anyMatch(d -> d.getName().equals(discipline)))
        .collect(Collectors.toList());
}
}

```

### 4.3 JdbcWorkshopService.java (extrait — bookWorkshop)

```

@Override
public void bookWorkshop(Workshop workshop, CommunityMember member) {
    if (workshop == null || member == null
        || workshop.getId() == null || member.getId() == null) return;

    String sql = "INSERT INTO booking (workshop_id, member_id, booking_date,
payment_status) "
        + "VALUES (?, ?, NOW(), 'PENDING')";
    try (Connection conn = ConnectionManager.getConnection();
        PreparedStatement ps = conn.prepareStatement(
            sql, Statement.RETURN_GENERATED_KEYS)) {
        ps.setLong(1, workshop.getId());
        ps.setLong(2, member.getId());
        ps.executeUpdate();
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) {
                Booking b = new Booking(workshop, member);
                b.setId(keys.getLong(1));
            }
        }
    }
}

```

```
        member.addBooking(b);
    }
}
} catch (SQLException e) {
    throw new RuntimeException("Erreur bookWorkshop", e);
}
}
```

## 6. Conclusion

L'intégration de la base de données MySQL dans l'application ArtConnect repose sur une architecture claire en 4 couches. Les services InMemory\* ont été intégralement remplacés par leurs équivalents JDBC sans modification des contrôleurs JavaFX.

Bilan des fichiers modifiés ou créés :

- Modifiés : 8 entités (ajout du champ id), ConnectionManager, ServiceProvider
- Créés : 5 DAOs JDBC, 5 services JDBC, DatabaseConfig
- Inchangés : tous les contrôleurs JavaFX, toutes les vues FXML, les interfaces de service et DAO

Ce découplage garantit que la couche de persistance peut être remplacée (ex. PostgreSQL, ORM) sans toucher à l'interface graphique.